

Governance-Driven Agentic Coding

一家 AI Native 公司的治理方式

一种由 Owner 掌舵、由 Agent 执行，并通过授权、留痕、独立挑战、只读汇报与学习闭环进行治理的软件开发方法。

不是让几个 AI 一起写代码，
而是把它们组织成一个可治理、可追溯、可替换的生产系统。

· BUILD	· REPORT	· CHALLENGE	· TEACH	· EXTERNALIZE
---------	----------	-------------	---------	---------------

VERSION 1.4 | 2026-07-30 | 基于跨项目实践持续迭代 | 小红书 @就很菜 ·
veggiedesu

第 01 节 执行摘要

核心定义

治理驱动的 Agent 开发，是一种由 Owner 设定授权与风险边界，由 Managing Director Agent 负责交付，并通过独立挑战、组织记录、只读信息投影和人类最终问责来治理的软件开发方式。

它解决的不是「怎样让更多 Agent 一起工作」，而是一个更根本的委托问题：当人类无法亲自掌握全部技术细节时，如何扩大 AI 的自主执行能力，同时不失去目标、判断、风险控制和最终责任。

这套系统的长期价值不依赖某个特定模型。Codex、Claude 或 GPT 只是当前岗位承担者；真正需要保存的是岗位职责、权限边界、证据、决策记录、升级规则和验收制度。

一句话概括：模型可以替换，岗位和制度必须延续。

五个核心职能

- ◆ **Build** — Managing Director 负责创造并交付项目价值。
- ◆ **Report** — Investor Office 负责把项目变化压缩成 Owner 可读的状态与汇报。
- ◆ **Challenge** — Strategic Board 负责重大方向与高风险决策的独立挑战。
- ◆ **Teach** — Academy 把真实项目内容转化成 Owner 的理解力与判断力。
- ◆ **Externalize** — GTM & Narrative Office 把项目证据转化成可验证的外部表达。

第 02 节 第一性原理：委托不能等于失控

AI Coding 真正危险的，不只是 AI 写错代码。更深的风险是：Owner 在把执行委托出去时，也把控制权、理解力和责任一起交了出去。

因此，这套方法坚持三条底线：

- ◆ AI 的执行能力可以持续扩大，但 Owner 的最终判断权不能被转移。
- ◆ 人类不必参与每一个步骤，但必须理解关键假设、不可逆决定和剩余风险。
- ◆ 复杂工作可以被委托；对结果的责任不能被委托。

HUMAN-IN-COMMAND

目标不是把人留在每一步，而是把人从低价值执行中移出，同时把人保留在高价值判断中。

第 03 节 Investor Mandate 与组织宪法

Investor Mandate：单个项目的最高授权

Investor Mandate 不是普通 Brief，也不是技术规格。它定义了为什么投资、授权做什么、允许投入多少、哪些风险不能承担，以及什么结果才算成功。

组成

需要回答的问题

Vision

项目要创造什么现实价值

Scope / Non-scope

明确做什么，也明确不做什么

小红书 @就很菜 · veggiedesu

Capital	时间、Token、工具、外部服务与其他资源
Risk Appetite	可以接受的风险与绝对不能接受的风险
Success Criteria	用什么证据判断项目成功
Reserved Decisions	哪些决定必须由 Owner 拍板

Operating Constitution：跨项目不变的组织纪律

- ◆ 不得编造事实、进度、测试结果或业务影响。
- ◆ 事实、假设、风险和建议必须分开表达。
- ◆ 重大决定必须留下依据、替代方案与反对意见。
- ◆ 关键失败不能被摘要隐藏；不可逆操作必须升级。
- ◆ 任何 Agent 不能批准自己的最终交付。
- ◆ 重要知识不得只存在于聊天窗口；历史记录不能被覆盖。
- ◆ 不得未经授权扩大 Scope 或改变风险偏好。

第 04 节 组织结构与权限模型

组织分为两部分：能够影响 Product 的控制核心，以及只能读取事实、面向不同受众生成视图的只读派生层。

岗 位	核 心 职 责	权 限 边 界	主 要 产 出
Owner / Investor	Vision、Capital、Risk Appetite、Legal Risk、最终验收	保留最终决定权与责任	Investor Mandate、接受 / 暂停 / 终止决定

Managing Director	计划、架构、委托、质量、记录、交付	可自主处理授权范围内的普通工程决定	Product、Project Ledger、Decision Log
Strategic Board	按 mandate 对重大方向、长期取舍与高风险决策独立挑战	只建议和二审；不管理、不决策、不执行	Board Package、独立意见、MD disposition
Investor Office	站在 Owner 一侧汇报项目真实状态	只读；不管理 MD、不修改 Product	Project State、Dashboard、Executive Brief
Academy	把项目事实转化为 Owner 的理解与判断力	只读；不得改变开发顺序	项目驱动教学、Capability Ledger
GTM & Narrative	把已发生事实转化为对外叙事	只读；不得为了故事反向要求产品	Demo、Case Study、Narrative

联邦式 Strategic Board

每个项目与组合办公室分别保留自己的战略挑战语境和可见记录。当前可以由 Claude 在 Owner-visible Chrome 对话中承担建议与二审，但 Claude 只是岗位的当前承担者，不拥有管理、决策或实施权。一个 mandate 的建议不会自动成为另一个 mandate 的要求；Advice、MD disposition、Owner decision、implementation 与 publication status 必须分别记录。

临时执行角色

Specialist Agents 与 Red-team Agents 由 MD 按任务动态生成。它们不是长期部门；任务结束即可退出。每个临时角色必须有明确的 mission、scope、inputs、output、completion criteria 和 escalation rule。

第 05 节 标准运行循环

- 02MD 把授权转化为最短可验证闭环，并建立任务依赖、测试策略和升级点。
- 03MD 动态创建 Specialist Agents，分别完成研究、实现、测试、文档或安全工作。
- 04所有 Agent 沟通、决定、失败和修复强制留痕。
- 05MD 完成第一轮自检；这属于质量控制，不等于独立审计。
- 06根据风险等级触发专项红队或 Strategic Board。
- 07普通问题由 MD 修复；超出授权或风险偏好的问题升级给 Owner。
- 08Owner 按 Mandate 进行最终验收：接受、附条件接受、返工、暂停或终止。
- 09项目结束后，沉淀组织记忆、Owner 能力与对外叙事资产。

按风险分级审查

风险等级	典型触发	审查与决策
低风险 · 可逆	局部改动、容易回滚、影响边界清晰	MD 自检
中风险 · 跨模块	接口变化、数据处理、多模块依赖	1-2 个专项红队
高风险 · 不可逆	安全、法律、资本、重大架构、上线门槛	多红队 + Strategic Board
超出授权	改变目标、扩大 Scope、突破预算或风险偏好	Owner 决定

审查原则：红队数量本身没有价值；新的攻击角度、证据来源或专业能力才有价值。红队可以由 MD 按任务生成，但高风险审查必须包含至少一个不由 MD 创建、不共享 MD 上下文的独立视角（如 Strategic Board 或独立红队）——否则组织会落入自己定义的同源自审。

第 06 节

Organizational Memory：组织记忆

关键差异

只读 Agent 读取的不是「MD 告诉我的」，而是「Codex 留下的组织记录」。

所有 Agent 内部沟通、决策与修复都必须留下日志；留痕是制度要求，不依赖任何岗位的自觉。关键 Decision Log 采用 append-only 纪律：不能覆盖历史；如需修订，也必须完整保留版本、时间和原因。

记录层	核心问题	主要内容
Project Ledger	项目现在在哪里	当前阶段、已完成 / 进行中、风险与阻塞、下一里程碑、关键证据
Decision Ledger	项目为什么走到这里	决定、背景、替代方案、证据、反对意见、理由、后续验证
Raw Logs	具体发生了什么	Agent 沟通、命令、测试、失败、修复与时间顺序
Executive Summary	Owner 现在需要知道什么	变化、健康度、风险、阻塞、需拍板事项

原始日志用于追溯；摘要用于恢复工作与降低认知负担。摘要可以被重写，但不能替代或删除原始记录。

组织记忆的价值：以后能够回答「为什么当时选 A，而不是 B」，而不是依赖某个模型模糊地回忆。

第 07 节

四条信息流

这套组织不是简单的上下级关系。它同时运行四条方向不同、权限不同的信息流。

小红书 @就很菜 · veggiedesu

Delivery Flow

Investor Mandate → MD + Specialists → Product

回答：项目怎样完成。

Reporting Flow

Material change → Mandatory log → Investor Office → Project State /
Dashboard → Owner

回答：项目现在怎样。

Learning Flow

Project Facts → Academy → Owner Capability

回答：Owner 因项目学会了什么。

Market Flow

Project Evidence → GTM Office → Demo / Case / Narrative → External Audience

回答：外部为什么应该理解和相信。

共同事实层： Project Ledger + append-only Decision Log。

第 08 节 三个只读办公室

Investor Office：Owner 的项目秘书与汇报办公室

Investor Office 直属 Owner，不是 MD 的秘书，也不负责管理项目。当主线里程碑、计划、风险、阻塞或重要决定发生实质变化时，MD 主动通知它。它读取 Project Ledger 与 Decision Log 的变化，生成 Owner 每天会看的管理信息。

◆ **Project State** — 项目当前阶段、健康度、下一里程碑与阻塞。

◆ **Dashboard** — 面向 Owner 的高密度管理视图。

◆ **Executive Brief** — 重要变化、重大决定、风险与建议。

◆ **Owner Decisions Required** — 只有 Owner 才能决定的事项及截止时间。

小红书 @就很菜 · veggiedesu

Academy：项目驱动的 Owner 学习系统

Academy 只读取项目真实发生的内容。它不脱离项目讲抽象课程，也不能为了教学要求 MD 改变开发顺序。它回答：今天发生了什么、为什么这样做、背后的机制是什么、以后 Owner 可以如何独立判断。

学习目标

普通 AI Coding 优化 how to finish the project; Academy 进一步优化 how the human becomes more capable after every project。

GTM & Narrative Office：事实驱动的外部表达

GTM Office 根据项目证据生成 Demo、Case Study、面试叙事与传播材料。它不能把计划写成已实现，不能隐藏失败，也不能为了故事好听反向要求 Product 改变。

最高纪律： Narrative follows evidence. 叙事必须跟随事实。

第 09 节 一个项目留下五类复利资产

在这套方法下，代码只是一个结果。同一次投入会同时生成五类可复用资产。

- ◆ **Product** — 可以运行、测试、演示或继续迭代的项目交付物。
- ◆ **Organizational Memory** — 强制留痕、完整保留的 Decision Log、错误集与制度演化记录。
- ◆ **Project State + Dashboard** — Investor Office 为 Owner 生成的可持续管理信息。
- ◆ **Owner Capability** — Academy 沉淀的概念理解、风险判断和独立决策能力。
- ◆ **Demo / Case / Narrative** — GTM Office 生成的外部表达、职业与商业资产。

项目在推进，组织在积累记忆，Owner 在积累判断力，成果也在积累外部可见性。

第 10 节 Lean 纪律、失效模式与衡量

组织复杂度不能超过项目复杂度。新增岗位必须解决一个已经出现的问题，而不是预防一个想象中的问题；保留岗位必须持续证明其价值高于协调成本。

新增岗位的成立条件

- ◆ 带来新的信息来源。
- ◆ 带来新的专业能力。
- ◆ 带来新的独立视角。
- ◆ 带来新的监督机制。

常见失效模式

失效模式	表现
Prompt cosplay	只有 CEO、Reviewer 等称号，没有权限、输入输出和升级规则。
同源自审	同一个模型生成方案，再让几个相同上下文的角色轻微反对。
Owner 放弃责任	以「不懂技术」为由，不定义验收和风险边界。
口头汇报替代记录	Investor Office 只听 MD 总结，不读取原始组织记录。

Narrative capture 为了故事或 Demo 反向影响产品事实和优先级。

岗位永久化 岗位建立后不再验证价值，组织不断膨胀。

建议衡量的组织 KPI

- ◆ Owner 读取原始日志的时间是否下降。
- ◆ 重大风险是否更早被发现和升级。
- ◆ 红队发现的有效问题比例与误报比例。
- ◆ Decision 被推翻或返工的原因是否可追溯。
- ◆ Project State 与实际交付是否一致。
- ◆ Academy 是否提高 Owner 的独立判断能力。
- ◆ GTM 是否让外部受众更准确地理解项目价值。

第 11 节 实践记录：三次治理干预

制度的有效性不体现在流程图，而体现在它实际拦截了什么。以下三条来自本方法当前运行的真实项目；表述已做脱敏。

记录 · 壹

测量中途变更，整批作废

在一轮由 Owner 亲自执行的验收评审中，MD 中途修改了部分评审材料的表述。独立红队判定：评审进行中变更评审对象，污染的是整批结果，而不只是被改动的部分。该轮评审被整体标记为无效并按冻结版本重做；原始记录、改动内容与作废理由完整保留，不删除。

审核人拒绝凭摘要出具通过

一次里程碑复核中，Strategic Board 拒绝仅凭一段实现摘要出具通过结论，理由是：如果审核环节接受口头摘要作为凭证，控制链顶端反而成为全链最弱的一环。该轮结论被明确降级为「仅对所述设计的判断」，与经独立核实的结论严格区分。

证据顺序从被相信变为可证明

红队要求：关键评审材料在交付评审人之前，必须先形成版本快照（如一次 Git commit）；评审产出必须位于其后的快照中。材料与结论的先后顺序从此是可验证的事实，而不是流程声明。

共同点：三次干预拦下的都不是模型的能力缺陷，而是流程在压力下想省事的那个瞬间。

第 12 节 Practice → Evidence：可审计的治理学习闭环

版本化学习

应用项目提供事实与经验，但不能自行改写共同方法；未经验证的 hypothesis 不得提前成为 canonical method。

阶段	必须留下的记录	控制边界
Practice	原始 observation、friction、失败或 near miss	描述发生了什么，不先抽象成规则
Evidence	可复现步骤、版本快照、反例、适用边界	区分事实、来源证明与尚未验证项

Proposed Change	规则提案、迁移影响、回滚方案、版本候选	提案不等于方法已改变
Decision	接受、修改、拒绝或继续验证及其理由	内部 patch/minor 可由组合治理办公室批准；重大原则与公开发布归 Owner
Versioned Method	版本号、Change Log、证据引用、迁移说明	只有获批变更进入正式版本
Re-test	来源场景与迁移场景的复测结果	被反证时回滚、缩小边界或降级为 hypothesis

何时启动治理复盘

- ◆ 同一摩擦在两个项目重复出现。
- ◆ 单个项目出现重大控制缺口或 near miss。
- ◆ 角色或授权边界不清导致返工。
- ◆ 实践反证既有方法声明。
- ◆ 出现可复现、可迁移的新治理机制。
- ◆ Agentic automation 暴露新的监督或回滚需要。
- ◆ 重大里程碑提供足够的新证据。

版本边界

级别	典型变化	批准边界
Patch	错字、澄清、引用或非实质性证据补充	不改变权限、控制门或行为要求
Minor	向后兼容的新机制、触发器、记录类型或低风险控制	不改变核心决策权、风险偏好或人类决定边界

Major 核心原则、权责、强制控制或 human decision Owner 决定
boundary 改变

第 13 节 能力声明、结论来源与派生完整性

跨项目实践表明，治理不仅要保存证据，还要约束证据能够支持什么结论。以下三条规则是 domain-neutral 的；应用实现可以不同，但不能越过同一事实边界。

1. Capability Claim Attestation

Agent-executed、AI-executed、自动化或模型评估等能力声明，必须绑定实际 runtime、provider、execution event 与相关配置或产出证据。接口、fixture、mock、provider adapter 或 agent-ready 结构只证明准备状态，不能证明 Agent 已执行该案例。

2. Conclusion Provenance

human-only、human-approved 或 reviewer-approved 不能靠标签自声明。结论必须能从 exception 状态与 human review chain 验证或重算，至少能够回答：谁看了什么证据、作出什么决定、是否仍有未解决例外。

3. Derivation Integrity

输入或 raw hash 只证明该对象的身份与未变性，不能证明 downstream draft、建议或结论由它派生。关键链路必须通过可重放转换、canonical equality、可测试的等价规则或 derivation manifest 连接 raw → draft → conclusion。

RiskFirewall AI — Product Risk Review (Complex Instruments · Second Line) 与 RiskFirewall AI — Risk Control Assurance (Transactions, Processes & AI Actions · Third Line) 为本方法的两条应用线。

命名边界： RiskFirewall AI 是组合品牌，不是第四个仓库，也不证明 live AI execution、自动决策、生产 firewall、已部署控制系统或两个应用具有相同实现。

已保留限制： provider interface 不等于 live execution；source authenticity 与 semantic sufficiency 仍可能需要人类复核；hash 不等于语义派生；应用项目的证据成熟度必须分别陈述。

第 14 节 最低可行实施清单

最低可行版本不需要十几个 Agent。先建立一个有写权限的生产核心、一个战略挑战角色、三个只读视图和五类基本记录。

- 01 写出一页 Investor Mandate。
- 02 明确 Owner、MD 与 Strategic Board 的权限边界。
- 03 让 MD 持续维护 Project Ledger 与 append-only Decision Log。
- 04 规定哪些项目变化会主动触发 Investor Office。
- 05 建立低、中、高三级风险审查协议。
- 06 确保 Academy 与 GTM 只读，不反向修改 Product。
- 07 定义最终 Acceptance Gate 和可接受的剩余风险。
- 08 建立 Practice → Evidence → Proposed Change → Decision → Versioned Method → Re-test 记录。
- 09 审查 capability claim、human conclusion provenance 与关键派生链。

Acceptance Gate

验收层	判断内容
Functional	交付物是否能够工作，关键测试是否通过
Assurance	证据、红队、风险记录与失败处理是否完整
Investor	结果是否仍符合 Investor Mandate 与风险偏好

最终状态可以是 **Accepted**、**Accepted with residual risks**、**Conditional acceptance**、**Rework required**、**Paused** 或 **Terminated**，而不应只有「完成」。

这套方法不是为了让 AI 看起来像一家公司。它要解决的是：如何把概率性的、不完全可靠的模型，组织成一个可替换、可追溯、可审查、可学习的生产系统。

你不是在雇佣某个模型。
你在建立一套模型可替换、事实可追溯、风险可升级、
Owner 始终掌舵的制度。

附则 — 本文总结的是一套正在两个应用项目中实践、反证与迭代的方法，不将其表述为已经完成行业验证的标准，也不把未发布候选、私有实现或接口准备状态描述为公开能力。